Dossier Técnico

Arquitectura de Estilos y Preprocesamiento en Interfaces de Usuario

Módulo Profesional: Desarrollo de Interfaces (Código 0488) — 2º DAM
Marco Curricular: Decreto 68/2012 de la Comunidad de Madrid

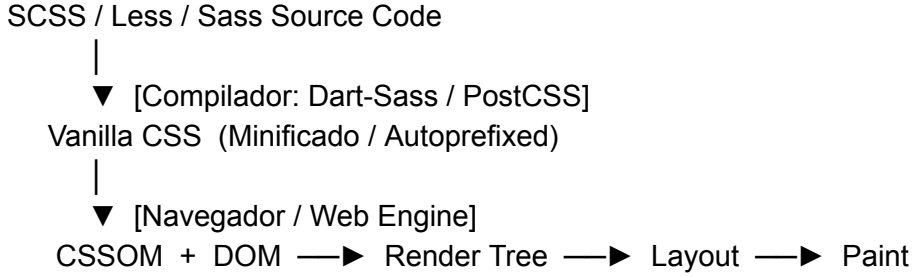
- **RA 2:** Genera interfaces de usuario a partir de documentos basados en lenguajes de marcas, evaluando la sintaxis y aplicando estilos consistentes.
  - *CE 2.b:* Se han aplicado hojas de estilo para modificar la apariencia y disposición de los componentes.
  - *CE 2.c:* Se han empleado selectores avanzados, herencia y cascada para optimizar la definición visual.
- **RA 4:** Diseña interfaces gráficas respetando criterios de usabilidad, accesibilidad y capacidad de adaptación responsive.
  - *CE 4.a:* Se han aplicado directrices de diseño responsive adaptando la interfaz a diferentes resoluciones de pantalla.
  - *CE 4.c:* Se han estructurado los estilos visuales mediante arquitecturas escalables y mantenibles.

1. Base Teórica Técnica: Evolución, Preprocesamiento y Arquitecturas

1.1 CSS Nativo vs. Preprocesadores (Sass, SCSS, Less)

El diseño moderno de interfaces exige desacoplar presentación, estructura y lógica. Mientras que CSS tradicional históricamente carecía de modularidad y reusabilidad avanzada, los preprocesadores introdujeron compilación estática a CSS estándar:

- **Sass (Syntactically Awesome Style Sheets):**
  - *Sintaxis Indentada (.sass):* Basada en indentación y saltos de línea, sin llaves ({}) ni puntos y comas (;). Es más concisa pero incompatible con la sintaxis estándar de CSS copy-paste.
  - *SCSS (.scss - Sassy CSS):* Superset estricto de CSS3. Cualquier CSS válido es SCSS válido. Soporta anidamiento (*nesting*), variables locales/globales ($color), mixins con directivas @mixin/@include, funciones aritméticas y módulos estructurados con @use y @forward (reemplazando el obsoleto @import).
- **Less (.less):**
  - Basado en JavaScript (ejecutable tanto en Node.js como directamente en cliente vía less.js). Emplea @ para variables (ej. @primary-color: #007bff;). Aunque facilitó el auge inicial de Bootstrap 3, su adopción industrial ha declinado frente al estándar consolidado de Sass/Dart-Sass.
- **CSS Moderno Nativo:**
  - La especificación CSS actual ya implementa *CSS Custom Properties* (var(--token)), nativas y dinámicas en tiempo de ejecución (resolubles vía DOM y JavaScript), y soporte nativo para *CSS Nesting* en navegadores modernos. Sin embargo, SCSS sigue dominando en compilación estática, tipado de funciones, bucles de generación de utilidades y gestión de bundles en entornos CI/CD.



1.2 Patrones de Diseño Arquitectónico en CSS

Cuando las aplicaciones cliente escalan, la colisión de especificidad (*specificity wars*) y la falta de encapsulación degradan el mantenimiento:

Metodología / Patrón	**Enfoque**	**Estructura de Clases**	**Casos de Uso Recomendados**
BEM *(Block Element Modifier)*	Nombres explícitos para eliminar la cascada profunda y dependencias de etiquetas.	.card (Bloque) .card__title (Elemento) .card--featured (Modificador)	Sistemas de componentes aislados, design systems en SCSS/CSS Modules.
OOCSS *(Object-Oriented CSS)*	Separa estructura (layout) de piel (tema/color).	.btn (estructura base) + .btn-primary (tema visual)	Librerías de componentes UI reutilizables sin duplicar reglas de posición.
SMACSS *(Scalable & Modular Architecture)*	Categorización de reglas en cinco capas jerárquicas.	Base, Layout, Module, State, Theme	Grandes portales corporativos y proyectos monolíticos.
Utility-First	Clases atómicas con propósito único compiladas bajo demanda.	.flex .items-center .p-4 .bg-blue-600	Interfaces rápidas, equipos ágiles, stacks con Tailwind CSS.

1.3 Comparativa de Frameworks de Estilos para Interfaces

- **Tailwind CSS (Utility-First):**
  - *Motor:* PostCSS + compilador JIT (*Just-In-Time*).
  - *Ventajas:* Cero CSS no utilizado (*purge* automático), altísima velocidad de iteración, evita nombrar clases intermedias, consistencia estricta vía tokens de diseño (tailwind.config.js).
  - *Inconvenientes:* Marcado HTML denso, curva de asimilación de nombres de utilidades.
- **Bootstrap 5 (Component-Based):**
  - *Motor:* SCSS modular compilable, sin dependencias de jQuery.
  - *Ventajas:* Catálogo extenso de componentes listos para producción (modales, navbars, offcanvases), sistema de rejilla flexbox/CSS grid consolidado.
  - *Inconvenientes:* Interfaces visualmente reconocibles sin personalización profunda, mayor peso base de bundle.
- **CSS Modules / Styled Components (Enfoques scoped en SPAs):**
  - *Mecanismo:* Generación de hashes únicos por clase (.button_a7x9f) para garantizar aislamiento absoluto de ámbito (*scope*) a nivel de componente visual (React/Vue/Angular).

2. Ejemplo Funcional Paso a Paso: Arquitectura Modular con SCSS Moderno (@use) y BEM

Construcción de un componente de interfaz interactivo: un **DataCard** accesible, responsive y tematizable con variables semánticas.

Estructura de Directorios (Patrón 7-1 Simplificado)

```
styles/
├── abstracts/
│   ├── _variables.scss
│   └── _mixins.scss
├── components/
│   └── _data-card.scss
└── main.scss
```

Código Fuente

styles/abstracts/_variables.scss

SCSS

```scss
// Tokens de diseño semánticos
$font-stack: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;

$color-surface: #ffffff;
$color-surface-muted: #f8fafc;
$color-border: #e2e8f0;
$color-text-main: #0f172a;
$color-text-muted: #64748b;
$color-primary: #2563eb;
$color-primary-hover: #1d4ed8;
$color-danger: #dc2626;

$radius-md: 0.5rem;
$spacing-sm: 0.75rem;
$spacing-md: 1.25rem;

$transition-base: 200ms ease-in-out;
```

styles/abstracts/_mixins.scss

SCSS

```scss
@use "variables" as v;

// Mixin responsive: Mobile-first media query
@mixin respond-to($breakpoint) {
  @if $breakpoint == "tablet" {
    @media (min-width: 768px) { @content; }
  } @else if $breakpoint == "desktop" {
    @media (min-width: 1024px) { @content; }
  }
}

// Mixin flexbox centrado
@mixin flex-between {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

styles/components/_data-card.scss

SCSS

```scss
@use "../abstracts/variables" as v;
@use "../abstracts/mixins" as m;

.data-card {
  background-color: v.$color-surface;
  border: 1px solid v.$color-border;
  border-radius: v.$radius-md;
  padding: v.$spacing-md;
  font-family: v.$font-stack;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
  transition: transform v.$transition-base, box-shadow v.$transition-base;

  &:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  }

  // Elemento: Cabecera del componente
  &__header {
    @include m.flex-between;
    margin-bottom: v.$spacing-sm;
  }

  // Elemento: Título
  &__title {
    margin: 0;
    font-size: 1.125rem;
    font-weight: 600;
    color: v.$color-text-main;
  }

  // Elemento: Indicador de estado (Badge)
  &__badge {
    padding: 0.25rem 0.5rem;
    font-size: 0.75rem;
    font-weight: 700;
    text-transform: uppercase;
    border-radius: 9999px;
    background-color: v.$color-surface-muted;
    color: v.$color-text-muted;
```

```
    &--active {
      background-color: rgba(v.$color-primary, 0.1);
      color: v.$color-primary;
    }

    &--critical {
      background-color: rgba(v.$color-danger, 0.1);
      color: v.$color-danger;
    }
  }

  // Elemento: Cuerpo con adaptación responsive
  &__content {
    color: v.$color-text-muted;
    font-size: 0.875rem;
    line-height: 1.5;

    @include m.respond-to("tablet") {
      font-size: 1rem;
    }
  }

  // Modificador de variante: Tema destacado
  &--highlighted {
    border-left: 4px solid v.$color-primary;
  }
}
```

styles/main.scss

```
SCSS
@use "components/data-card";

// Normalización básica de la interfaz
body {
  margin: 0;
  padding: 2rem;
  background-color: #f1f5f9;
}
```

index.html (Instanciación del componente accesible)

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Módulo 0488 - Implementación BEM / SCSS</title>
  <link rel="stylesheet" href="dist/main.css">
</head>
<body>

  <article class="data-card data-card--highlighted" aria-labelledby="card-title-01">
    <header class="data-card__header">
      <h2 id="card-title-01" class="data-card__title">Servicio de Autenticación</h2>
      <span class="data-card__badge data-card__badge--active">Operativo</span>
    </header>
    <section class="data-card__content">
      <p>Latencia media de respuesta: <strong>42ms</strong>. Todas las réplicas en Docker Swarm responden sin anomalías de memoria.</p>
    </section>
  </article>

</body>
</html>
```